Scaling Laws at Small Scale: Revisiting *Chinchilla* for Tiny Transformer Language Models

Ahmad Sayad – EN.705.742.81.SP26

Project Code: https://github.com/asayad1/SmallScalingLaws

Abstract:

Classical scaling-law results, such as those from *Hoffmann et al. (2022)* (henceforth referred to as *"Chinchilla"*), indicate that for a fixed training compute budget, increasing the number of training tokens is often more effective than increasing model capacity for optimal allocation of compute during training. This paper investigates whether the same relationship holds for small decoder-only transformer language models. This report investigates how limited compute should be allocated between model size and training data by asking whether better validation performance is obtained from training smaller models on more tokens or larger models on fewer tokens when total compute is held approximately constant. To examine this tradeoff, we perform a controlled training sweep over a family of compact transformer architectures across multiple compute budgets, using the approximation $C \approx 6ND$, where N is the trainable parameter count and D is the number of training tokens. For each budget, we compare multiple model-data allocations under two learning-rate schedule regimes: a matched schedule that scales with training duration and a fixed schedule with a common horizon. Performance is evaluated primarily using validation cross-entropy loss, with multiple random seeds used to reduce variance and improve the robustness of estimated frontiers. Beyond extracting empirical frontiers, we fit a smooth scaling surface to characterize how the compute-optimal allocation varies across budgets. The results indicate that the qualitative structure of scaling laws persists at small scale, with clear compute-optimal tradeoffs between model size and data. Under the fixed schedule, the fitted exponents ($\alpha \approx 0.27, \beta \approx 0.33$) are close to values reported by *Chinchilla*, and the optimal token-to-parameter ratio falls in the range of approximately 35-50 at higher compute budgets, compared to the Chinchilla heuristic of ~20. In contrast, the matched schedule produces substantially different allocations, favoring much smaller models with ratios often exceeding 150, indicating strong sensitivity to optimization conditions. These results suggest that while *Chinchilla*-style scaling laws remain broadly applicable, accurate estimation at small scale requires careful control of training dynamics, and the compute-optimal data-to-parameter ratio is higher in this regime.

Introduction:

Large language models have shown consistent performance improvements as model size, dataset size, and total training compute increase. Classical scaling-law work, particularly *Hoffmann et al. (2022)*, suggests that under a fixed training compute budget, allocating compute toward more training tokens is often more effective than increasing model capacity. This result has become an important guide for modern language model training, but it is less clear whether the same relationship holds in the regime of small decoder-only transformers, where optimization behavior and experimental design choices may have a larger influence on the observed frontier – that is, the best achievable validation loss under a given compute constraint.

This report investigates the compute-optimal tradeoff between model size and training data for autoregressive transformer language models at small scale. Specifically, for a fixed training compute budget, we study which combination of parameter count and training tokens minimizes validation loss. This allows insight into whether smaller models trained on more tokens outperform larger models trained on fewer tokens. This question is naturally framed through scaling-law analysis, which characterizes how

loss varies as a function of model size, dataset size, and compute, and in turn informs how limited pretraining resources should be allocated.

To study this tradeoff, we conduct controlled training sweeps across multiple compute budgets and estimate the empirical compute-optimal allocation of model size and training tokens using validation loss. A key focus of this work is the role of learning-rate scheduling. We examine how sensitive the compute-optimal tradeoff is to the choice of schedule, since apparent scaling behavior can be misleading when models trained for different durations are compared under mismatched optimization schedules. This issue is particularly pronounced at small and intermediate scales, where experimental results are more sensitive to design choices and may obscure underlying scaling behavior.

Accordingly, this report has two objectives: first, to estimate the empirical compute-optimal allocation of parameters and training tokens within a compact transformer family, and second, to evaluate how sensitive this allocation is to the choice of learning-rate schedule. By comparing validation loss across iso-compute sweeps (i.e., groups of runs with approximately constant training compute) and analyzing both empirical and fitted frontiers, we aim to clarify how training compute should be distributed between model capacity and data exposure in this small-model setting. Our hypothesis is that *Chinchilla*-style scaling laws hold at small scale, and that the optimal token-to-parameter ratio is less than the value implied by *Chinchilla* due to the more compact model architectures.

Background & Related Work:

Early empirical scaling-law work by *Kaplan et al. (2020)* showed that language-model loss follows smooth power-law trends with respect to model size, dataset size, and compute. A central conclusion of that study was that, under a fixed training compute budget, larger models trained on fewer tokens could be more efficient than smaller models trained for longer, corresponding to relatively low token-to-parameter ratios on the order of 1-2 tokens per parameter. This perspective strongly influenced early large-scale language-model training, encouraging aggressive scaling of model size. In practice, this meant prioritizing increases in parameter count over proportional increases in dataset size.

This view was later reevaluated by *Hoffmann et al. (2022)* in the *Chinchilla* study, which argued that many large language models were substantially undertrained. Their results suggested that compute-optimal training requires scaling model size and training tokens more proportionally, with an optimal ratio of approximately 20 tokens per parameter – an order of magnitude higher than the ratios implied by *Kaplan et al.* Under this regime, smaller models trained on more data can outperform much larger models trained on too few tokens at comparable compute. This shifted attention away from parameter count alone and toward the balance between model capacity and data exposure, and *Chinchilla* remains a central reference for compute-optimal pretraining.

More recent work has focused directly on the tension between *Kaplan*-style and *Chinchilla*-style conclusions. *Pearce and Song (2024)* show that part of the discrepancy can be explained by differences in parameter counting and by the smaller-scale regime of the original *Kaplan* analysis, recovering behavior closer to *Chinchilla* once those factors are accounted for. *Porian et al. (2024)* further identify three concrete sources of disagreement: last-layer compute accounting, warmup duration, and scale-dependent optimizer tuning. Together, these results demonstrate that estimates of compute-optimal performance can shift materially when optimization settings and accounting conventions are not controlled consistently across scales. This sensitivity motivates the present study's emphasis on carefully constructed iso-compute sweeps and on evaluating the robustness of the observed compute-optimal frontier under different learning-rate schedule regimes.

Problem Formulation & Scaling Law Framework:

Training a language model under limited resources can be viewed as an allocation problem. Under a fixed training compute budget, one must decide how to distribute compute between model size and data exposure: either training a larger model for fewer steps or a smaller model on more tokens. The central question in this work is therefore: for a fixed compute budget, what combination of parameter count and training-token count minimizes validation loss?

We consider the standard causal language modeling objective. Given a token sequence $x_1, x_2, \ldots, x_{t-1}$, the model predicts the next token x_t. Training minimizes negative log-likelihood, and performance is evaluated using validation cross-entropy loss on held-out text. This provides a consistent scalar metric for comparing models trained under different allocations of compute.

Following the *Chinchilla* scaling-law framework, the final validation loss of a language model can be modeled as:

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}$$

where N is the number of model parameters, D is the number of training tokens, and E represents the irreducible loss of the data distribution. The terms A/N^{α} and B/D^{β} capture excess loss due to finite model capacity and finite data, respectively. The exponents α and β determine the relative returns to scaling model size and data. Larger α implies that model-capacity limitations diminish more quickly with increasing N, while larger β implies that additional data yields diminishing returns more rapidly.

The *Chinchilla* paper estimates $\alpha \approx 0.34$ and $\beta \approx 0.28$ for large-scale transformer models, indicating earlier diminishing returns for decreasing loss with model size than with data. These values imply that model size and training tokens should scale nearly proportionally, leading to the widely cited heuristic of approximately 20 training tokens per parameter. In this work, we fit the same parametric form at small scale to evaluate whether similar exponents and token-to-parameter ratios emerge for compact models.

To relate this loss model to resource constraints, we use the standard approximation for total training compute:

$$C = 6ND$$

where C is measured in floating-point operations. This approximation reflects the cost of forward and backward passes in dense transformer training and serves as a hardware-agnostic proxy for compute. Rearranging gives:

$$D = \frac{C}{6N}$$

which defines the fundamental tradeoff. Under fixed compute, increasing model size reduces the number of tokens that can be processed, and vice versa. The compute-optimal allocation can be derived by minimizing the loss under this constraint. Substituting $D = C/(6N)$ into the loss yields:

$$L(N, D) = E + AN^{-\alpha} + B\left(\frac{C}{6N}\right)^{-\beta}$$

Differentiating with respect to N and solving for the optimum gives scaling relations of the form:

$$N^{*}(C) \propto C^{\frac{\beta}{\alpha+\beta}}, \qquad D^{*}(C) \propto C^{\frac{\alpha}{\alpha+\beta}}$$

When $\alpha \approx \beta$, this ratio is approximately constant with respect to compute, implying that model size and data should scale proportionally. When $\alpha > \beta$, the ratio increases with compute, favoring relatively more data at larger scales. When $\alpha < \beta$, the ratio decreases, favoring relatively larger models.

Empirically, this tradeoff is studied using iso-compute sweeps. For a fixed compute budget C, an iso-compute sweep consists of a set of runs with different (N, D) pairs that satisfy the same compute constraint. In the (N, D) plane, these runs lie along a curve defined by $D = C/(6N)$. Comparing validation loss across such runs isolates the effect of allocation, allowing identification of the most effective use of compute at that budget.

The primary object of interest is the compute-optimal frontier. For each budget C, the best-performing configuration on the corresponding iso-compute sweep defines an empirical optimum. Collecting these optima across budgets yields functions:

$$N_{opt}(C), \quad D_{opt}(C)$$

which describe how the optimal allocation evolves with increasing compute. The ratio:

$$\frac{D_{opt}(C)}{N_{opt}(C)}$$

provides a direct empirical estimate of how heavily models should be trained relative to their size, and can be compared to the theoretical predictions implied by α and β.

Finally, although this framework is expressed in terms of parameters and tokens, training is implemented in optimizer steps. Changing the token budget changes the number of training steps, which alters how a run interacts with its learning-rate schedule. Models trained for many steps may complete a full decay schedule, while those trained for fewer steps may terminate before the schedule has fully decayed. As a result, the observed compute-optimal allocation may depend not only on the parameter-data tradeoff, but also on whether the optimization schedule is appropriately matched to the training duration. This motivates evaluating multiple learning-rate schedule regimes as part of the experimental design.

Experimental Setup:

1. Model Architecture

All models in the sweep follow a GPT-style decoder-only Transformer architecture using the pre-norm formulation. Each model consists of token embeddings, learned positional embeddings, a stack of Transformer blocks, a final layer normalization, and a linear output head. The output projection shares weights with the token embedding matrix.

Each Transformer block contains a multi-head causal self-attention sublayer and a feed-forward MLP sublayer, both with residual connections. Attention is computed using scaled dot-product attention with a causal mask to enforce autoregressive behavior. The MLP uses a $4\times$ expansion ratio with GELU activation. Layer normalization is applied before each sublayer, and dropout with rate $\rho = 0.1$ is applied after attention, after the MLP, and to the input embeddings. Weights are initialized following the GPT-2 convention (with all linear and embedding weights drawn from a normal distribution $N(0, 0.02)$ and biases initialized to zero), with residual projection layers scaled by $1/\sqrt{2n_{layer}}$ to stabilize training across depth.

For scaling law analysis, the parameter count N excludes embedding parameters. Specifically, N includes all attention, MLP, and normalization parameters, while excluding both token embeddings and positional

embeddings. This convention follows the *Chinchilla* study, which argues that embedding parameters do not contribute to model capacity in the same manner as internal network parameters. Although embedding parameters are excluded from the parameter count, the embedding dimension still determines the size of all internal representations and therefore directly controls model capacity.

The model grid spans 21 configurations, varying in both depth and width. Depth ranges from 2 to 5 layers, while width ranges from 16 to 96 embedding dimensions, with a fixed number of attention heads ($n_{head} = 4$). This yields models with approximately 11,000 to 600,000 non-embedding parameters (see Table A1 in the appendix for the model configurations). The grid is intentionally denser for smaller models, ensuring that the compute-optimal configuration is well bracketed at lower compute budgets, while larger models provide coverage at higher budgets.

2. Data & Tokenization

Training is conducted on the *WikiText-103* corpus. The dataset is tokenized at the byte level, treating each UTF-8 byte as an individual token. This results in a fixed vocabulary of size $V = 256$ and avoids the complexity of sub-word tokenization schemes such as BPE. The dataset sizes are summarized in Table 1:

Split	Number of tokens
Train	~540 million
Validation	~1.1 million
Test	~1.3 million

Table 1: Number of tokens in the training, testing, and validation splits.

Training sequences of length $T = 128$ are sampled uniformly at random from the training corpus. Validation and test losses are computed on fixed batches drawn sequentially from their respective splits.

3. Training Procedure

All models are trained using the *AdamW* optimizer with $\beta_1 = 0.9$ and $\beta_2 = 0.95$, and weight decay of 0.1. Gradients are clipped to a maximum norm of 1.0. Dropout is fixed at 0.1 across all layers. Learning rate is fixed at $\eta_0 = 6 \times 10^{-4}$ for all models regardless of size. Training was done on an *NVIDIA L40S* GPU.

A. Learning Rate Schedule

The learning rate follows a linear warmup followed by a cosine decay. During the warmup phase, the learning rate increases linearly from zero to the peak value η over W steps. After warmup, it decays according to a cosine schedule down to a minimum value $\eta_{min} = 0.1\eta$, after which it remains constant. The warmup duration is defined as:

$$W = \max(1, \lceil 0.05 * S_{decay} \rceil)$$

allowing schedules to scale proportionally with training length.

Two learning-rate schedule regimes are considered. In the matched schedule regime, the decay horizon is set equal to the total number of training steps for each run, ensuring that each model experiences a complete warmup and decay cycle. In the fixed schedule regime, all models at a given compute budget share a common decay horizon, set to the median number of training steps across model sizes at that budget.

4. Iso-Compute Sweep Design

Experiments are organized as iso-compute sweeps across nine compute budgets. The budgets are listed in Table 2.

Budget ID	Compute C (FLOPs)
1	1.2×10^{12}
2	2.0×10^{12}
3	3.2×10^{12}
4	5.0×10^{12}
5	8.0×10^{12}
6	1.3×10^{13}
7	2.0×10^{13}
8	3.2×10^{13}
9	5.0×10^{13}

Table 2: The compute budgets throughout the iso-sweeps.

For each model with parameter count N, the number of training tokens is determined by the compute constraint:

$$D = \frac{C}{6N}$$

This ensures that all runs at a given budget consume approximately equal compute. The number of training steps is then:

$$S = \left\lfloor \frac{D}{B \times T} \right\rfloor$$

where $B = 16$ and $T = 128$, corresponding to 2,048 tokens per step. Configurations with fewer than 50 or more than 30,000 training steps are excluded to avoid unstable or impractically long runs. Each configuration is trained with three random seeds under both schedule regimes.

5. Evaluation & Aggregation

Validation loss is evaluated periodically during training and averaged over a fixed set of validation batches. The final validation loss for each run is defined as the loss at the final training step. Results are aggregated across the three random seeds, and the mean validation loss is used for all subsequent analysis. In addition to final validation loss, the best validation loss observed during training is also tracked.

For each compute budget, the model configuration achieving the lowest mean validation loss is selected as the empirical optimum. Collecting these optima across budgets yields the empirical compute-optimal frontier.

6. Scaling Surface Fitting

To estimate scaling-law parameters, we fit the *Chinchilla* parametric model

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}$$

to the observed data. To improve numerical stability, the coefficients A and B are parameterized in log-space:

$$A = e^{a}, \quad B = e^{b}$$

and optimization is performed over (E, a, α, b, β). This guarantees that A and B are non-negative. The parameters are estimated using *L-BFGS-B* to minimize mean squared error between predicted and observed losses. To reduce bias, data points with very low training steps or infeasibly low D/N ratios are

excluded from the fit. The fitted surface is used to derive a parametric estimate of the compute-optimal frontier, which is compared against the empirical frontier.

Results:

1. Fitted Scaling Law Parameters

We first fit the *Chinchilla* parametric scaling surface

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}$$

independently for the fixed and matched learning-rate schedule regimes. Each configuration (C, N) is averaged over three seeds and treated as a single observation. After filtering out configurations with fewer than 300 training steps or a token-to-parameter ratio $D/N < 10$, each fit uses 129 observations out of 152 total configurations per regime. The fitted parameters are shown in Table 3:

Parameter	Fixed Schedule	Matched Schedule	*Chinchilla*
E	0.753	0.872	1.69
A	20.10	32.95	–
α	***0.273***	***0.368***	***0.34***
B	122.52	58.21	–
β	***0.333***	***0.265***	***0.28***
RMSE	0.0732	0.0643	–

Table 3: Fitted scaling law parameters. *Chinchilla's* irreducible loss is not directly comparable due to differences in tokenization and corpus.

The two schedule regimes produce similar fit quality but qualitatively different scaling exponents. Under the fixed schedule, $\beta > \alpha$, while under the matched schedule $\alpha > \beta$. This implies fundamentally different compute-optimal allocations depending on the schedule regime. The quality of the fit is shown in Figure 1:

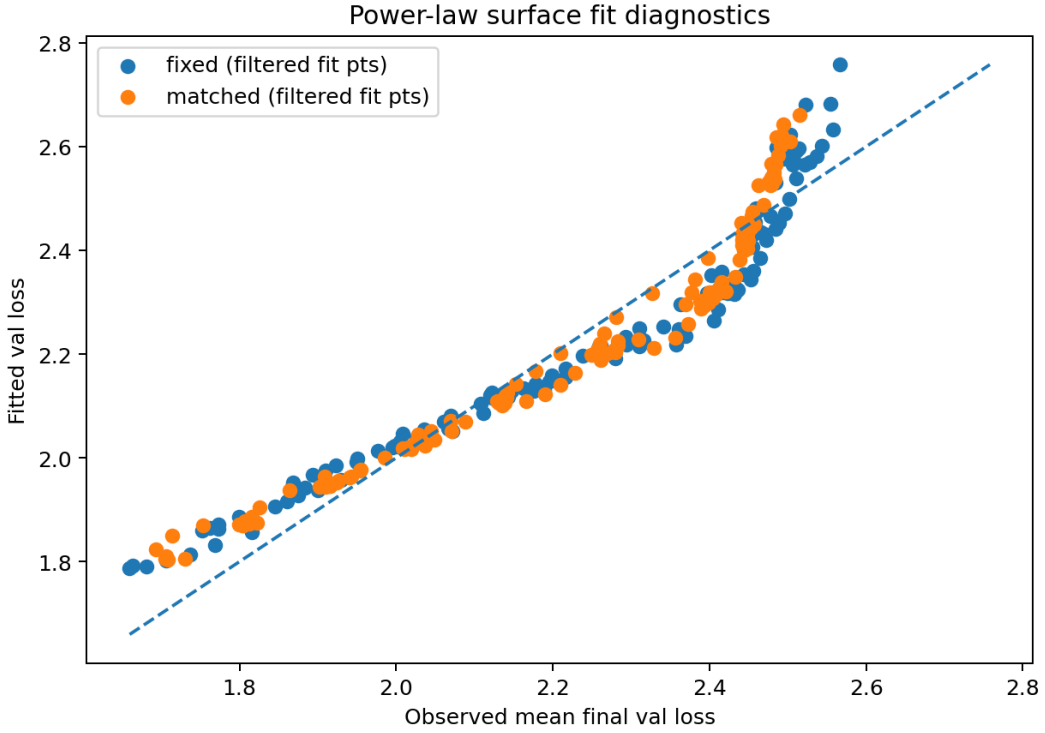
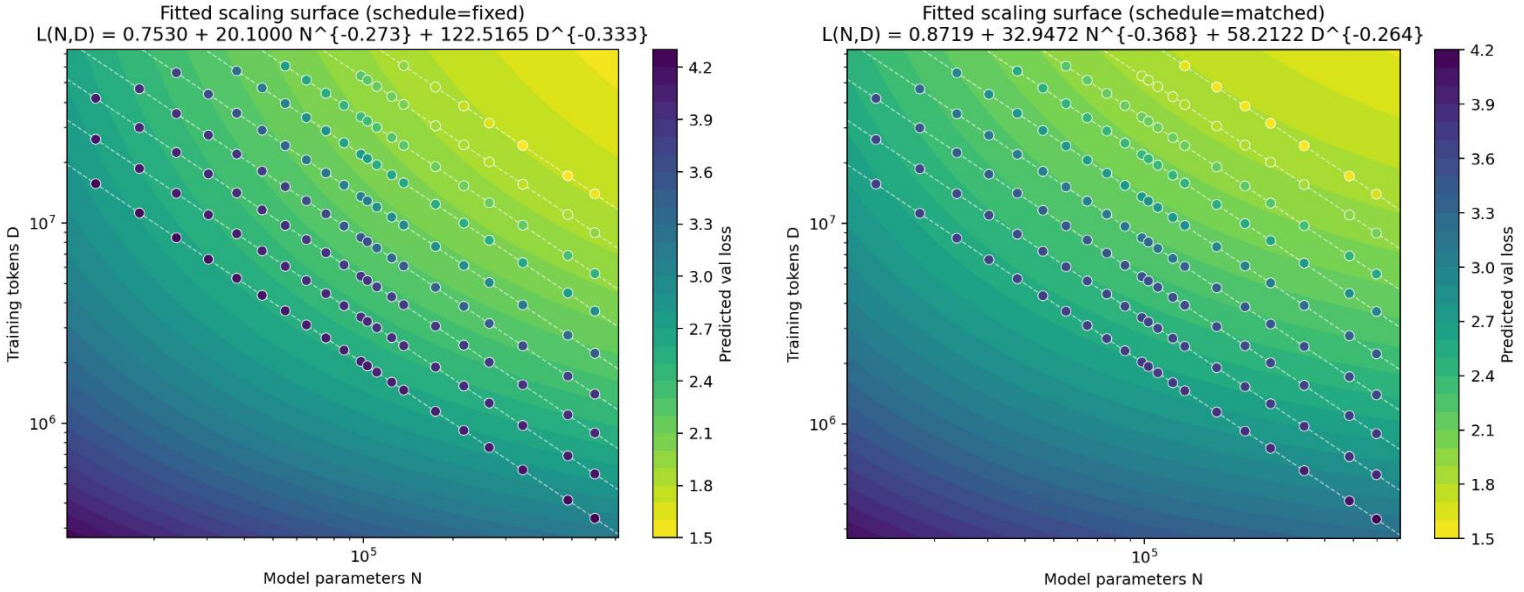


Figure 1: Predicted vs. observed validation loss for the fitted scaling surface. The dashed line is $y = x$; Points on the line are perfect predictions, points above mean the surface underpredicts the observed loss, and points below mean it overpredicts.

The RMSE of 0.073 (under the fixed regime) and 0.064 (under the matched regime) corresponds to roughly 3% relative error given the loss range of ~1.6 to ~2.6. The matched regime fit is slightly tighter, consistent with its more homogeneous schedule structure. The fitted scaling surfaces are visualized in Figures 2 & 3:



Figures 2 & 3: Contour plots of the fitted loss surface over a log-log grid of N and D. The white dashes are the iso-compute contours; i.e. the line $D = C/(6N)$ for each of the 9 budgets. The dots within are aggregated results used to fit the surface, with its filled color being the observed validation loss.

The fixed-schedule surface shows a relatively flat ridge where model size and data contribute comparably, while the matched-schedule surface has a steeper gradient along the N axis, reflecting the higher α.

2. Derived Scaling Exponents

From the fitted values of α and β, we compute the implied scaling relationships. The results are compiled in Table 4:

Quantity	Formula	Exponent (Fixed)	Exponent (Matched)	*Chinchilla*
Optimal Model Size	$N^* \propto C^{\beta/(\alpha+\beta)}$	0.549	0.418	0.46
Optimal Data Size	$D^* \propto C^{\alpha/(\alpha+\beta)}$	0.451	0.582	0.54
Token-to-Parameter Trend	$D^*/N^* \propto C^{(\alpha-\beta)/(\alpha+\beta)}$	-0.099	0.163	0.097

Table 4: Scaling exponents derived from the fitted values of α and β. Note, *Chinchilla* doesn't directly report a token-to-parameter trend value. It was computed from the reported α and β values.

The fixed schedule implies a decreasing token-to-parameter ratio with compute, while the matched schedule implies an increasing ratio. The exponents can be interpreted in terms of how training compute should be allocated between model size and data. Under the fixed schedule, the fitted exponents are: $N^* \propto$

$C^{0.549}$, and $D^* \propto C^{0.418}$. This implies that as compute increases, a larger fraction of additional compute should be allocated to scaling model size rather than training data. In contrast to *Chinchilla*, where data scaling dominates slightly, the fixed-schedule fit suggests that model capacity is the primary limiting factor at this scale.

Under the matched schedule, the fitted exponents are $N^* \propto C^{0.418}$, and $D^* \propto C^{0.582}$. This allocation more closely resembles the *Chinchilla* result, with data scaling receiving a larger share of compute. However, the gap between the exponents is larger than in *Chinchilla*, implying a stronger preference for increasing data relative to model size.

3. Empirical Compute-Optimal Frontier

The empirical compute-optimal frontier was constructed by selecting, at each compute budget, the configuration with the lowest mean validation loss across the three seeds training was performed on. See Figures A2 & A3 in the appendix for the Iso-compute sweeps at each compute budget for fixed, and matched schedules respectively.

The compute-optimal frontier from empirical data for the fixed schedule regime is given in Table 5:

Compute Budget (FLOPs)	Model	N	D	D/N	Loss
$\mathbf{1.2 \times 10^{12}}$	gpt_2l_60d	111,120	1,800,192	16.2	2.485
$\mathbf{2.0 \times 10^{12}}$	gpt_2l_60d	111,120	3,000,320	27.0	2.447
$\mathbf{3.2 \times 10^{12}}$	gpt_2l_60d	111,120	4,800,512	43.2	2.397
$\mathbf{5.0 \times 10^{12}}$	gpt_2l_64d	124,672	6,684,672	53.6	2.259
$\mathbf{8.0 \times 10^{12}}$	gpt_2l_64d	124,672	10,696,704	85.8	2.120
$\mathbf{1.3 \times 10^{13}}$	gpt_3l_72d	217,224	9,975,808	45.9	1.995
$\mathbf{2.0 \times 10^{13}}$	gpt_3l_72d	217,224	15,345,664	70.6	1.868
$\mathbf{3.2 \times 10^{13}}$	gpt_4l_80d	342,240	15,585,280	45.5	1.752
$\mathbf{5.0 \times 10^{13}}$	gpt_4l_96d	484,416	17,203,200	35.5	1.660

Table 5: The compute-optimal frontier for the fixed schedule regime. See Table A1 in the appendix to reference model architecture differences.

The fixed-schedule frontier favors moderately sized models. At the three lowest budgets, the 2-layer, 60-dimension model (111,000 params) is optimal. The frontier then shifts to deeper models as compute increases: 3-layer 72-dimension at 13-20 TFLOPs, then 4-layer models at 32-50 TFLOPs. The D/N ratio ranges from 16 to 86, with a non-monotonic pattern. It increases as compute budget ranges from 1.2 TFLOPs to 8 TFLOPs, then drops as the frontier shifts to deeper, wider models.

And likewise, for the matched schedule regime the compute-optimal frontier was calculated as:

Compute Budget (FLOPs)	Model	N	D	D/N	Loss
$\mathbf{1.2 \times 10^{12}}$	gpt_2l_32d	37,760	5,298,176	140.3	2.477
$\mathbf{2.0 \times 10^{12}}$	gpt_2l_44d	64,592	5,160,960	79.9	2.442
$\mathbf{3.2 \times 10^{12}}$	gpt_2l_36d	45,936	11,612,160	252.8	2.369
$\mathbf{5.0 \times 10^{12}}$	gpt_2l_44d	64,592	12,902,400	199.8	2.249
$\mathbf{8.0 \times 10^{12}}$	gpt_2l_44d	64,592	20,643,840	319.6	2.128
$\mathbf{1.3 \times 10^{13}}$	gpt_2l_60d	111,120	19,499,008	175.5	2.008
$\mathbf{2.0 \times 10^{13}}$	gpt_2l_60d	111,120	29,999,104	270.0	1.902
$\mathbf{3.2 \times 10^{13}}$	gpt_3l_64d	174,656	30,537,728	174.8	1.799

$\mathbf{5.0 \times 10^{13}}$	gpt_4l_80d	342,240	24,350,720	71.2	1.694

Table 6: The compute-optimal frontier for the matched schedule regime. See Table A1 in the appendix to reference model architecture differences.

The matched-schedule frontier consistently selects smaller, shallower models than the fixed schedule at the same budget. At the first seven budgets, the optimal model is a 2-layer architecture with 32,000-111,000 parameters. The D/N ratios are much higher, ranging from 71 to 320, with large non-monotonic fluctuations (e.g., jumping from 79.9 at 2 TFLOPs to 252.8 at 3.2 TFLOPs, then back down). This volatility reflects the fact that the matched schedule strongly favors small models that train for many steps with a full cosine decay, and the frontier is sensitive to which small model happens to be at a sweet spot in the discrete grid. In both training regimes, no frontier point lies at the boundary of the model grid, indicating that the optimum is well bracketed.

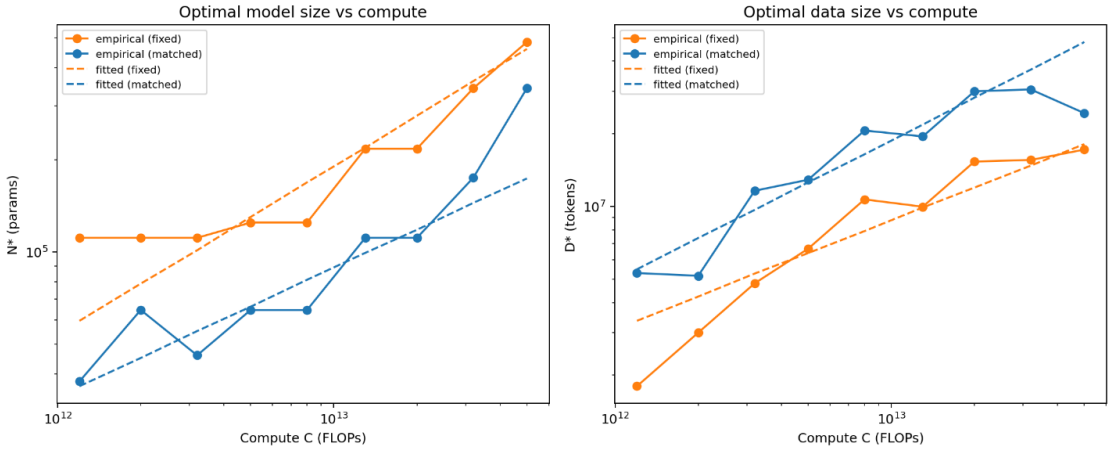


Figure 4: Optimal model size & data vs. compute

Figure 4 compares the parametric frontier predicted by the fitted scaling surface against the empirical frontier points selected from the iso-compute sweeps. Consistent with the empirical frontier, the fixed-schedule fit predicts larger optimal models and slower growth in training tokens, while the matched-schedule fit predicts smaller optimal models trained on substantially more data.

Figure 5 shows how validation loss changes with model size under the fixed compute budgets, allowing us to identify the compute-optimal number of parameters for that budget. The empirical curves show the observed behavior under different learning-rate schedule regimes, while the dashed curve shows the prediction from the fitted scaling-law surface.

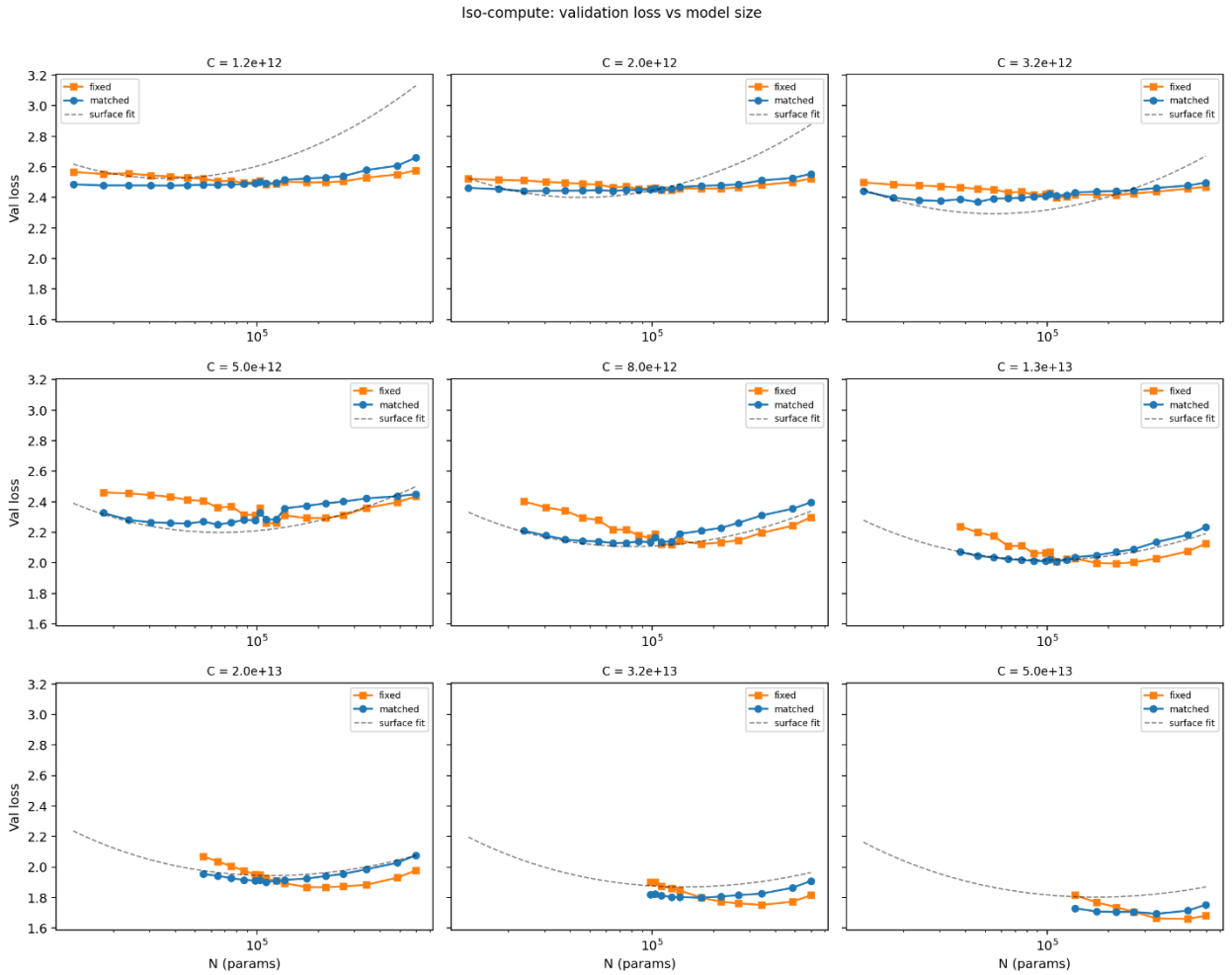


Figure 5: Validation loss vs model size across the compute budgets.

Figure 5 suggests that the dataset broadly follows the expected scaling-law behavior: as the compute budget increases, the best achievable validation loss decreases, and the compute-optimal model size shifts toward larger N. In the lower-compute slices, the loss curves are relatively flat and the minimum is harder to localize precisely, meaning several model sizes perform similarly well. At higher compute budgets, the U-shaped structure becomes clearer: smaller models appear under-parameterized, while very large models begin to suffer because the fixed compute budget leaves too little data per parameter.

4. Token-to-Parameter Ratio Comparison

Finally, the compute optimal token-to-parameter ratios for the fixed and matched training regimes are presented in Table 7:

	Fixed Schedule		Matched Schedule	
Budget	**Empirical D^*/N^***	**Fitted D^*/N^***	**Empirical D^*/N^***	**Fitted D^*/N^***
$\mathbf{1.2 \times 10^{12}}$	16.2	56.4	140.3	150.8
$\mathbf{2.0 \times 10^{12}}$	27.0	53.7	79.9	164.4
$\mathbf{3.2 \times 10^{12}}$	43.2	52.1	252.8	175.5
$\mathbf{5.0 \times 10^{12}}$	53.6	49.3	199.8	190.1
$\mathbf{8.0 \times 10^{12}}$	85.8	46.9	319.6	202.9
$\mathbf{1.3 \times 10^{13}}$	45.9	45.3	175.5	219.9

$\mathbf{2.0 \times 10^{13}}$	70.6	43.0	270.0	239.1
$\mathbf{3.2 \times 10^{13}}$	45.5	40.9	174.8	255.2
$\mathbf{5.0 \times 10^{13}}$	35.5	39.4	71.2	276.5
Mean	47.0	47.4	187.1	208.3

Table 7: Empirical & fixed token-to-parameter for both fixed and match schedule regimes.

In the fixed schedule, the empirical D/N ratio does not follow a fixed trend. It increases from 16 to 86 across the first five budgets, then decreases to 35 as the frontier shifts to deeper, wider models. The fitted ratio decreases monotonically from 56 to 39, reflecting the negative exponent found in the power-law relationship. Both empirical and fitted ratios are consistently 2-3x times above the *Chinchilla* value of 20.

The matched-schedule ratios are an order of magnitude above *Chinchilla*, with the empirical mean at 187 and the fitted mean at 208. The empirical ratio is highly volatile (71 to 320), while the fitted ratio increases monotonically from 151 to 277. These extreme values seem to be driven by the matched schedule's systematic bias for small, long-training models. Figure 6 visualizes these differences

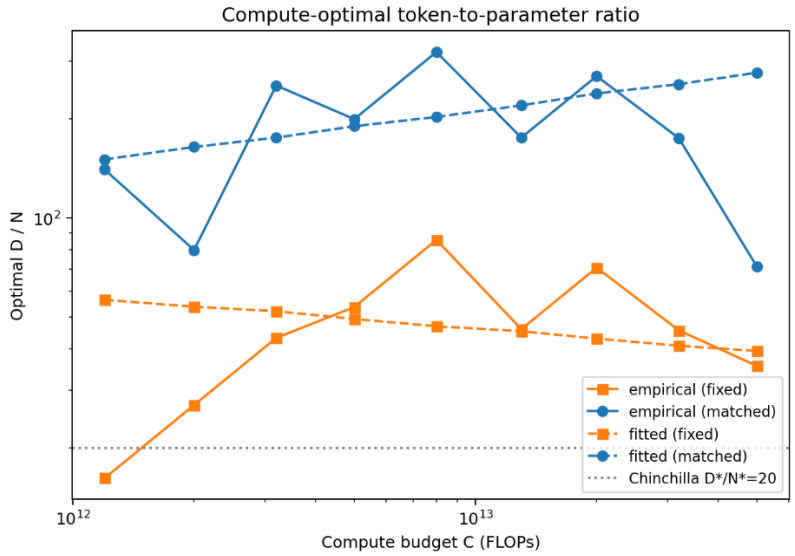


Figure 6: Empirical & Fitted D/N ratios.

Analysis:

1. Fixed Schedule Results Are Broadly Consistent With *Chinchilla*

The central result of this study is that the scaling-law exponents estimated under the fixed learning-rate schedule are broadly consist with the results in *Chinchilla*, despite operating in a substantially smaller model regime. While measurable deviations are present, such as the inversion of α and β, these differences are modest in magnitude and can plausibly be attributed to experimental constraints rather than a fundamental breakdown of the scaling-law relationship. The overarching result from *Chinchilla* that model size and data should scale at comparable rates still hold, even in this smaller regime. However, the fixed-schedule results suggest a modest shift toward allocating additional compute to model capacity, with model size growing slightly faster than data.

A. Individual Exponents Are Close to Chinchilla Results

Under the fixed schedule, we obtain $\alpha = 0.273$ and $\beta = 0.333$, compared to *Chinchilla's* $\alpha = 0.34$ and $\beta = 0.28$. The absolute deviations are small, on the order of 0.05-0.07. Moreover, the absolute difference

between the two exponents is nearly identical to the values reported in *Chinchilla*; the fixed-schedule gap $|\beta - \alpha| = 0.06$ which is approximately equal to *Chinchilla's* $|\alpha - \beta| = 0.06$.

A more stable indicator of scaling behavior is the sum $\alpha + \beta$, which determines the overall curvature of the loss surface and governs the combined rate of improvement as both model size and data are scaled. The fixed-schedule estimate yields $\alpha + \beta = 0.606$, comparable to *Chinchilla's* value of 0.62, a relative difference of 2.3%. This close agreement indicates that the overall steepness of the scaling surface is essentially preserved at small scale. Even if the relative contributions of model capacity and data differ slightly, the combined scaling behavior remains consistent with the large-scale regime. This conclusion is further supported by the derived scaling exponents for optimal allocation. Under the fixed schedule, the fitted relationships imply $N^*(C) \propto C^{0.549}$ and $D^*(C) \propto C^{0.451}$, whereas *Chinchilla* reports approximately $N^*(C) \propto C^{0.46}$ and $D^*(C) \propto C^{0.54}$. Although the fixed schedule slightly favors model growth over data, the exponents remain within approximately 0.10 of the Chinchilla values. Over the limited compute range explored here, this more likely corresponds to a modest shift in allocation rather than a fundamentally different scaling regime.

B. Empirical Iso-Compute Sweeps Validate the Tradeoff

Importantly, the empirical iso-compute sweeps in Figure 5 provide direct, model-independent evidence that the *Chinchilla* tradeoff remains valid at small scale. At compute budgets greater than 5.0×10^{12}, the loss curves as a function of model size exhibit clear U-shaped behavior, with well-defined interior minima. The most plausible explanation here is that models that are too small underutilize the available compute due to insufficient capacity, while models that are too large are effectively data-starved. The optimal configuration consistently lies between these extremes, confirming the existence of a genuine tradeoff between model size and data quantity. Moreover, frontier diagnostics from Figure A2 show that no optimal point lies at the boundary of the model grid at any budget, indicating that the experimental design successfully brackets the optimum.

C. Token-to-Parameter Ratios Are Elevated, but Converging

The empirical token-to-parameter ratios appear to broadly support the trend set by *Chinchilla* as well, albeit with some deviations. Under the fixed schedule, D^*/N^* ranges from 16.2 to 85.8, with a mean value of 47.0. While these values exceed the *Chinchilla* heuristic of approximately 20, they remain within the same order of magnitude and exhibit a downward trend at higher compute budgets, reaching 35.5 at 5×10^{13} FLOPs. This trend towards convergence suggests that the elevated ratios are partly a finite-scale effect rather than evidence of a different scaling law. While the fixed-schedule exponents suggest that additional compute should be allocated more heavily toward model size, the optimal token-to-parameter ratios remain higher than the *Chinchilla* heuristic. This is not contradictory. The elevated ratios reflect a higher baseline data requirement at small scale, while the scaling exponents describe how that allocation should evolve with increasing compute. Several factors likely contribute to this upward shift. Byte-level tokenization introduces higher entropy per token than sub-word tokenization, meaning that each token carries less predictive information and more data is required to achieve the same learning signal. Additionally, very small models may not fully utilize additional parameters without sufficient data, leading to slower decay of the model-capacity penalty at low N. Finally, the limited size and diversity of the *WikiText-103* corpus may cause the data penalty to saturate differently than in web-scale datasets. Taken together, these factors plausibly explain why the optimal token-to-parameter ratio is elevated at small scale while still trending toward *Chinchilla*-like values as compute increases.

2. The Matched Schedule Regime Produces Noisy Results

In contrast, the matched-schedule results exhibit behavior that is inconsistent with both *Chinchilla* and the fixed-schedule estimates. The fitted exponents and resulting D^*/N^* ratios are substantially larger, with empirical means of approximately 187 and fitted mean of approximately 208.

A. Systematic Bias

This discrepancy can be plausibly traced to a systematic optimization bias. Under the matched schedule, each run uses a cosine decay schedule matched to its own training length. Because smaller models process more tokens at fixed compute, they train for many more optimization steps than larger models. As a result, small models receive long, well-resolved schedules whilst larger models train for relatively fewer steps. This may create an uneven optimization landscape in which smaller models are consistently better optimized than larger ones.

The consequence of this imbalance is that the matched schedule artificially inflates the apparent benefit of training on more data while suppressing the apparent benefit of increasing model size. The resulting empirical frontier shifts toward small models with extremely high D/N ratios, and the fitted exponents reflect this distortion. The fitted token-to-parameter ratios, which range from approximately 150 to 277, are far outside the range observed in large-scale studies and do not converge toward *Chinchilla*-like values at higher compute. Instead, they increase monotonically with compute, contradicting both theoretical expectations and the empirical trends observed under the fixed schedule.

The fixed schedule, while not perfectly unbiased, provides a more controlled comparison by holding the schedule horizon approximately constant across models at a given compute budget. Although it introduces mild truncation for large models, it avoids the severe asymmetry present in the matched regime. For this reason, the fixed-schedule results should be considered the more reliable estimate of scaling behavior in this experiment.

3. Sources of Residual Bias

Even under the fixed schedule, several sources of bias remain that may influence the estimated exponents. First, the use of a fixed schedule horizon introduces truncation effects for larger models, which may terminate training before the learning rate has fully decayed. This can artificially inflate their final loss and bias the estimated model-capacity exponent downward. Second, the use of a single learning rate across all model sizes may not be optimal, as larger models typically require lower learning rates for stable convergence. Third, the model grid is relatively small and discrete, with only 21 configurations. This lessens the generalizability of the empirical frontier and limits the resolution of the scaling surface. Fourth, the compute range explored spans only 1.7 orders of magnitude, which makes the exponents more difficult to estimate precisely and increases sensitivity to noise. Additional factors, including the underestimated irreducible loss E, the use of byte-level tokenization, the limited corpus size, the presence of dropout, and residual variance across random seeds, may all contribute to deviations from the large-scale *Chinchilla* estimates.

Conclusions:

Taken together, the results suggest that the *Chinchilla* scaling law remains broadly applicable at small scale, albeit with a modest tilt toward model growth, with model size increasing slightly faster than data as compute increases. The fixed-schedule estimates recover similar exponent magnitudes, nearly identical values of $\alpha + \beta$, and clear empirical evidence of a compute-optimal tradeoff. The observed deviations (particularly the elevated token-to-parameter ratios) indicate that small models are more data-hungry than their large-scale counterparts, requiring more tokens per parameter to achieve optimal performance. These

differences can be explained by a combination of optimization bias, limited model capacity, and dataset characteristics. Importantly, the empirical ratios show signs of convergence toward *Chinchilla*-like values at higher compute, suggesting that the small-scale regime may approach the large-scale behavior as model size and compute increase. In contrast, the matched-schedule results are dominated by systemic optimization biases, and should not be interpreted as reflecting intrinsic scaling properties. Overall, these findings indicate that while scaling laws persist at small scale, their accurate estimation requires careful control of optimization conditions.

References:

[1] J. Hoffmann *et al*., "Training Compute-Optimal Large Language Models," *arXiv preprint arXiv:2203.15556*, 2022. doi: 10.48550/arXiv.2203.15556.

[2] J. Kaplan *et al*., "Scaling Laws for Neural Language Models," *arXiv preprint arXiv:2001.08361*, 2020. doi: 10.48550/arXiv.2001.08361.

[3] T. Pearce and J. Song, "Reconciling Kaplan and Chinchilla Scaling Laws," *arXiv preprint arXiv:2406.12907*, 2024. doi: 10.48550/arXiv.2406.12907.

[4] T. Porian, M. Wortsman, J. Jitsev, L. Schmidt, and Y. Carmon, "Resolving Discrepancies in Compute-Optimal Scaling of Language Models," *arXiv preprint arXiv:2406.19146*, 2024. doi: 10.48550/arXiv.2406.19146.

Appendix:

ID	Model Name	Layers	Embedding Dimensions	Number of Parameters
1	gpt_2l_16d	2	16	12,736
2	gpt_2l_20d	2	20	17,840
3	gpt_2l_24d	2	24	23,712
4	gpt_2l_28d	2	28	30,352
5	gpt_2l_32d	2	32	37,760
6	gpt_2l_36d	2	36	45,936
7	gpt_2l_40d	2	40	54,880
8	gpt_2l_44d	2	44	64,592
9	gpt_2l_48d	2	48	75,072
10	gpt_2l_52d	2	52	86,320
11	gpt_2l_56d	2	56	98,336
12	gpt_2l_60d	2	60	111,120
13	gpt_2l_64d	2	64	124,672
14	gpt_3l_48d	3	48	103,344
15	gpt_3l_56d	3	56	136,696
16	gpt_3l_64d	3	64	174,656
17	gpt_3l_72d	3	72	217,224
18	gpt_3l_80d	3	80	264,400
19	gpt_4l_80d	4	80	342,240
20	gpt_4l_96d	4	96	484,416
21	gpt_5l_96d	5	96	596,256

Table A1: The model configurations used in the sweep.

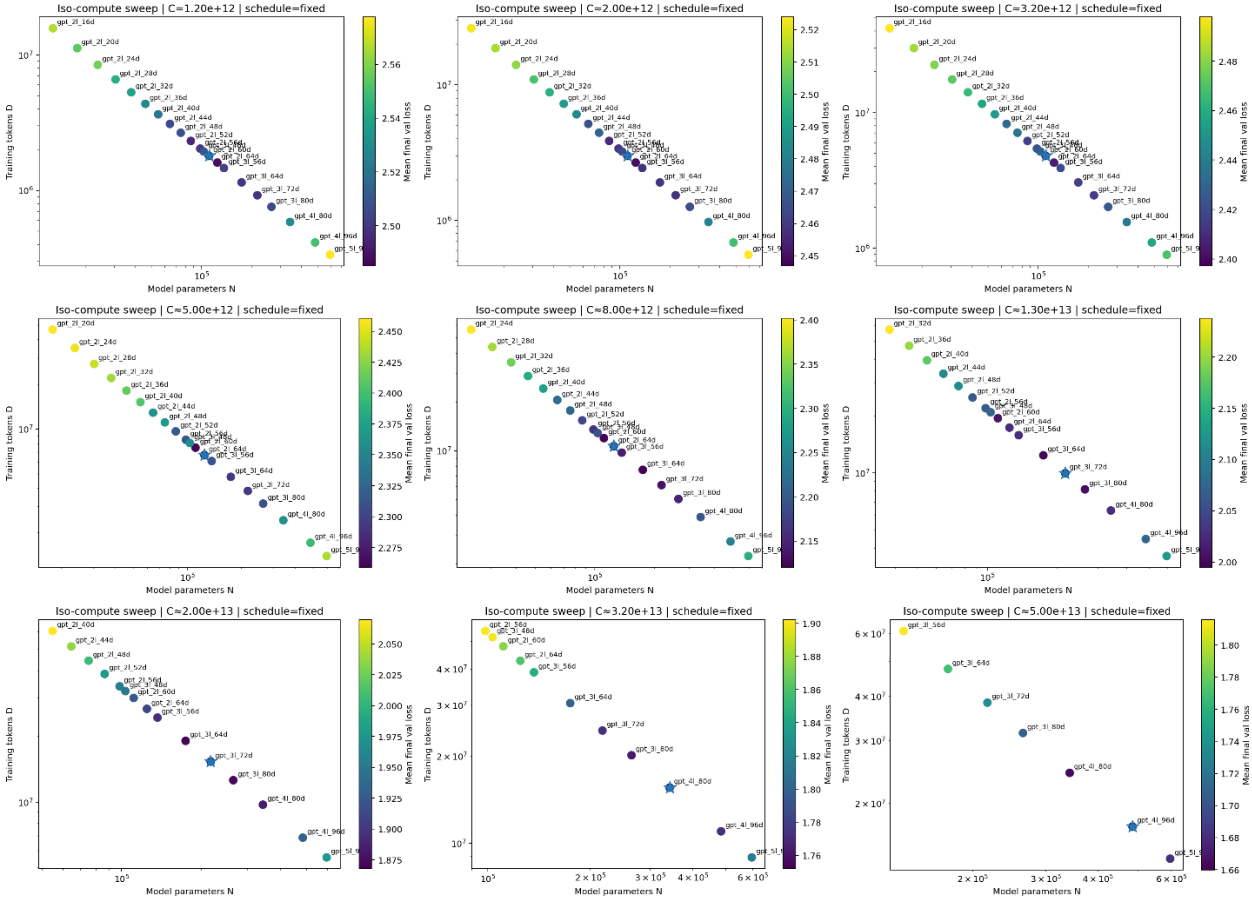


Figure A2: The Iso-compute sweep for each budget under the fixed regime. The optimal point that represents the frontier is represented with a star.

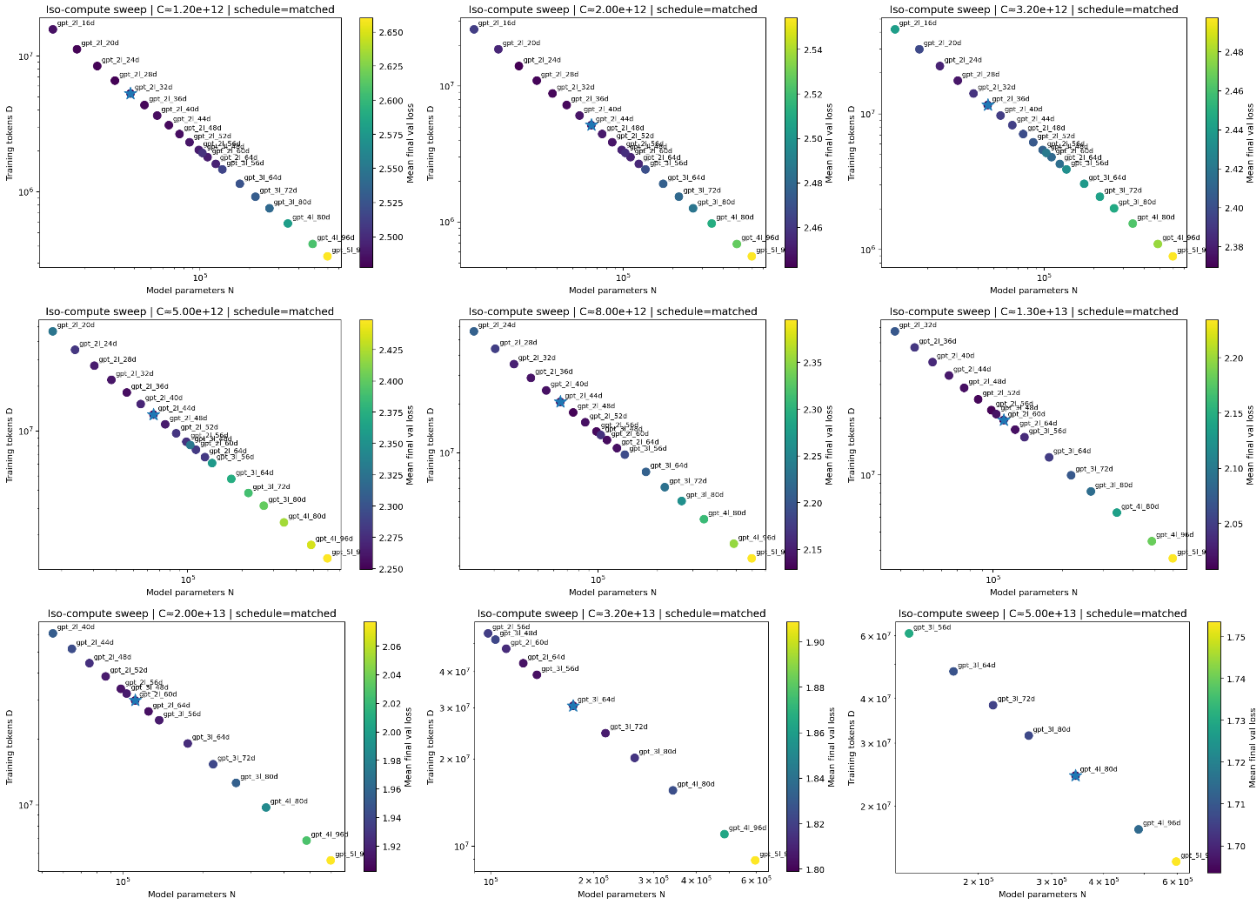


Figure A3: The Iso-compute sweep for each budget under the matched regime. The optimal point that represents the frontier is represented with a star.